

CHAINGUARD

技术方案说明书

面向制造企业的供应链中断应急决策系统。

本文所有数字均为实测值，测量方法与复现命令随各节给出。凡本仓库尚未验证的结论，一律标注为「未验证」而不写入指标表——这是本项目的对外纪律。

由 docs/技术方案说明书.md 自动生成 · 2026-07-24 · 内容与仓库同版本

目录

一、系统定位与要解决的问题

二、智能体整体架构

- 2.1 为什么是多 Agent，而不是一个大模型
- 2.2 决策链各环节
- 2.3 风险指数公式

三、性能量化测试数据

- 3.1 端到端决策时延
- 3.2 基线场景输出
- 3.3 消融实验
- 3.4 大模型数值幻觉拦截率
- 3.5 测试与验收规模
- 3.6 并发负载实测

四、功能模块设计

- 4.1 多租户与数据隔离
- 4.2 企业数据导入
- 4.3 权重校准闭环
- 4.4 节点健康判定：绝对红线与数据驱动阈值双轨取严
- 4.5 生产就绪能力

五、大模型的边界：只解释，不决策

- 5.1 边界
- 5.2 两道代码强制的闸门
- 5.3 实测：幻觉是真实存在的
- 5.4 严格性取舍

六、行业落地实施方案

- 6.1 分阶段路径
- 6.2 部署形态
- 6.3 已知限制（落地前必须知情）

七、可复现性

一、系统定位与要解决的问题

供应链中断（供应商停产、港口封闭、极端天气）发生时，企业的真实痛点不是「不知道出事了」，而是在**信息不全、多部门利益冲突的情况下，几小时内给出一个能落地、且事后说得清依据的决策。**

采购要保供、物流要时效、财务要控成本——三者目标天然冲突。人工协调依赖资深人员的经验，结果不可复现、责任不可追溯。

ChainGuard 把这个过程结构化为一个**可审计的决策链**：

库存风险前置触发 → 三 Agent 并行提案 → 约束穷举仲裁 → 经验回注 → 人工确认落库

关键设计取舍：**系统不替人做决定，它把决策依据摊开给人看。** 高风险决策强制人工确认，且每个数字都能现场复算。

二、智能体整体架构

2.1 为什么是多 Agent，而不是一个大模型

三个部门的目标函数**本来就不同**，不是一个模型「视角不够全面」的问题。把它们建模成独立效用函数的 Agent，冲突才能被显式表达、被仲裁，而不是被一个模型的隐式偏好抹平。

Agent	关注维度	实现
采购 Agent	供应商替代、采购周期、供货稳定性	<code>src/agents.py::ProcurementAgent</code>
物流 Agent	运输时效、路线风险、交付保障	<code>src/agents.py::LogisticsAgent</code>
财务 Agent	应急成本、订单毛利、违约损失	<code>src/agents.py::FinanceAgent</code>

三者共用 `src/game_model.py::PayoffModel` 计算效用，各自的权重不同——**Agent 的差异体现在目标函数上，不体现在话术上。**

2.2 决策链各环节

环节	模块	职责
风险前置触发	<code>src/inventory_monitor.py</code>	四因子加权算库存风险指数，越阈值才启动决策
提案生成	<code>src/agents.py</code>	三 Agent 各出策略组合与评分
约束求解	<code>src/constraint_solver.py</code>	3×3×3 全组合穷举，筛出可行解
辩论	<code>src/debate.py</code>	按数据条件触发反驳分支，最多 3 轮
仲裁	<code>src/arbitrator.py</code>	综合评分选定最终方案
经验回注	<code>src/feedback.py</code> 、 <code>src/vector_store.py</code>	TF-IDF 检索历史相似案例，带置信度
审计	<code>src/audit.py</code>	全链路留痕，判定是否需人工确认
表达层	<code>src/text_generator.py</code>	把已算好的结论翻译成中文（见第五节）

2.3 风险指数公式

```
inventory_risk_index = w1·shortage_urgency + w2·order_importance
                    + w3·transit_delay    + w4·external_event
```

四个分量均为 0–100，权重来自 `config/risk_weights.yaml`（专家先验），可由监督式校准替换（见 4.3）。分量定义见 `src/inventory_monitor.py::calculate_inventory_risk`：

分量	计算
shortage_urgency	由可支撑小时数线性映射，≤24h 记 100，≥72h 记 0
order_importance	$(1 - \min(\text{库存/关键订单需求}, 1)) \times 100$
transit_delay	$\min(\text{预计延误小时} / 72 \times 100, 100)$
external_event	事件严重度评分，直接取用

三、性能量化测试数据

3.1 端到端决策时延

固定演示场景，同一进程内连续 10 次完整决策链（含风险计算、三 Agent 提案、27 组合穷举、辩论、仲裁、经验检索）：

指标	数值
中位	1314.4 ms
P90	1324.2 ms
最小 / 最大	1292.6 / 1331.5 ms
均值 ± 标准差	1313.7 ± 12.2 ms

标准差 12.2 ms（相对波动 **0.93%**）说明决策链**完全确定性**，无随机成分——这是「同一输入必得同一输出、数字可现场复算」的直接证据。

与上一版（中位 1298.2 ms）相比增加约 16 ms（+1.2%），来自决策链新增的置信度校准与 challenger 批判环节。基线与消融的全部结论未变（见 3.2 / 3.3），即新增环节只增加了可解释性，没有改变决策输出。

复现：`python -c "from src.benchmark import _run_demo_isolated; _run_demo_isolated()"`

3.2 基线场景输出

指标	实测值
库存风险指数	70.25
可行方案组合数	27（3×3×3 全枚举共 27 种，该场景下全部满足约束）
辩论是否收敛	是
是否需人工确认	否（未越 80 阈值）

复现： `python -c "from src.benchmark import run_baseline; print(run_baseline())"`

3.3 消融实验

场景	风险指数	需人工确认	经验引用数
全模块启用	70.25	否	1（置信度 0.10）
关闭经验检索	70.25	否	0
库存降至 720	93.4	是	—
辩论未收敛	—	是	—

两条关键行为被验证：**风险越阈值自动升级为人工确认**；**辩论未收敛同样强制人工确认**——系统不会在内部意见不一致时替人拍板。

复现： `python -c "from src.benchmark import run_ablation; print(run_ablation())"`

3.4 大模型数值幻觉拦截率

见第五节，实测 88% → 0%。

3.5 测试与验收规模

项	实测
后端测试	863 passed / 5 skipped / 0 failed，耗时 230 s
前端单元测试	26 文件 / 79 用例全通过
端到端验收套件	10 套（data-import / calibration / risk-explanation / impact-scope / node-health / c3-onboarding / erp-integration / erp-mapping / experience / account），各套件独立库与端口，跳过数须与申报值一致否则判失败
数据库迁移	13 个版本，SQLite 与 PostgreSQL 双路径均在 CI 验证
代码规模	后端 130 个模块 / 24,326 行；REST 端点 125 个；前端页面 45 个
测试规模	97 个后端测试文件

CI 四条独立流水线：后端测试、PostgreSQL 迁移与约束验证（含备份恢复演练）、前端测试与构建、端到端验收门禁。

3.6 并发负载实测

复现： `python scripts/run_load_test.py --base-url <URL> --concurrency 1,4,16 --requests 60`

环境（数字只对这个环境成立）：单机 Windows，`uvicorn --workers 4`，SQLite，本机回环网络，演示租户（1 个可计算物料）。

端点	并发 1	并发 4	并发 16
<code>/healthz</code> (不鉴权不查库)	526 RPS / P50 1.8 ms	896 RPS / P50 3.9 ms	442 RPS / P50 26.3 ms
<code>/api/v1/risks</code>	290 RPS / P50 3.3 ms	757 RPS / P50 4.7 ms	464 RPS / P50 26.8 ms
<code>/api/v1/dashboard/kpis</code>	156 RPS / P50 6.3 ms	173 RPS / P50 22.3 ms	400 RPS / P50 32.2 ms
<code>/api/v1/dashboard/node-health</code> (最重只读)	81 RPS / P50 12.2 ms	171 RPS / P50 27.7 ms	299 RPS / P50 45.3 ms

720 个请求**零非 2xx 响应**。P95/P99 见脚本输出。

并发正确性：两个账号并发打 `/api/v1/auth/me`，检查 320 次，**身份串号 0 次**——请求态没有被跨请求污染（把租户/用户存进模块级全局变量是 FastAPI 应用的经典并发缺陷，单请求下永远正确，只有并发才暴露）。

复现：`python scripts/run_load_test.py ... --identity-check`

测量过程中的两个诚实说明：

1. 首轮测量出现过 P99 达 24~60 秒的尾延迟与吞吐塌陷。该现象在随后的 3 轮复现实验与一次冷启动专项实验中**均未再现**，且当时机器上有前端构建等进程在竞争。因此判定为环境性瞬时现象，**未认定为系统缺陷，也不据此下任何结论**。
2. 早期版本的脚本只预热一发请求，多 worker 下只暖到一个进程，其余进程的首次请求要付惰性加载代价（实测冷/暖差 2~4 倍），会被误读成“并发一上去就慢”。现已改为预热覆盖全部 worker。

仍未验证（不写入指标）：PostgreSQL 上的并发表现、生产规模数据集（本次演示租户仅 1 个可计算物料，node-health 的成本随物料数增长）、跨机器网络时延、长时间稳定性。上表**不能外推为生产 SLA**——它是一台开发机上的读路径基线，用途是回归对比，不是容量承诺。

四、功能模块设计

4.1 多租户与数据隔离

所有业务表带 `tenant_id`，查询层强制按租户过滤。在 PostgreSQL 上验证了业务键唯一性与租户隔离约束（`tests/test_phase5b_c2_postgres.py`，CI 独立 job）。

行级数据范围（全部/本部门及子部门/仅本人/自定义）落在 `src/webapi/data_scope.py`，依托 `departments` 部门树求值。未归属记录仅 `all` 范围可见——**宁可少放行，不可误放行**。

4.2 企业数据导入

`upload` → `preflight` → `confirm` → `execute` 四阶段，支持 CSV/Excel 与图片 OCR（RapidOCR + ONNX Runtime，本地推理，无需外部 API）。

- 识别结果**不自动生效**，低置信度显式标出，需人工确认资料类型
- 坏记录进隔离清单，不静默丢弃
- 导入批次可审计、可追溯

4.3 权重校准闭环

用**事前特征**做带 L2 正则的逻辑回归，系数转权重。与旧的相关系数法的区别是后者存在目标泄漏（拿事后结果当特征）。

准入门（`src/supervised_calibration.py`，任一不满足即拒绝并说明原因）：

条件	阈值
有效样本	≥ 80
正负类各	≥ 40
样本外 AUC	≥ 0.55
正系数	至少一个

「**拒绝校准**」是合法输出：拿不可信的权重替换专家先验，比不换更糟。校准建议不自动生效，必须管理员人工确认才写入租户配置，且落库时带追溯信息（样本量、方法、样本外 AUC、批准人）。

漂移监控以确认时的成功率为基线，跌幅越阈值触发告警。

4.4 节点健康判定：绝对红线与数据驱动阈值双轨取严

节点健康同时跑两条轨，取**较严者**：

- **绝对轨**：`config/thresholds.yaml` 的红/黄支撑时长（或租户获批的校准值），不受本批数据分布影响。
- **相对轨**：每次扫描由**本批风险分布**用「均值 + k·标准差」推导离群线，识别相对本批明显异常的节点。

为什么不能只用数据驱动那条——z-score 是相对量，单独使用有两个失效模式：① 全线告急时均值被抬高，真正的高危节点反而落回均值附近**不报警**；② 全线健康时批内分数最高的安全节点**被误判为高危**。绝对轨兜住①；相对轨额外设**绝对地板**（风险指数低于观察线时不参与升级）挡住②。两个失效模式都有测试锁死（`tests/test_threshold_calibration.py` 的期望值按均值与总体标准差手工推导，不是抄实现输出）。

样本不足（物料数 < 8）或分布无离散度时，相对轨**整体停用**并在界面如实说明「已回退为专家阈值，当前等价于仅绝对红线生效」——不把回退包装成智能阈值。每个节点同时给出「绝对轨判定」与「相对轨判定」，每条判据标注来源（`expert_default` / `calibrated`），判定过程可在界面逐条追溯。

4.5 生产就绪能力

能力	实现	验证
审计防篡改	每条审计记录带 <code>prev_hash</code> + <code>entry_hash</code> 构成哈希链	改一行/删一行均可检出
密钥轮换	静态加密 MultiFernet 多密钥；JWT 签发用当前密钥、验签依次尝试历史密钥	换钥不使在途令牌失效
跨进程作业	DB 持久化 + CAS 认领 + 租约回收 + 重试 + 优雅停机	多 worker 不重复执行
备份恢复	<code>scripts/backup-restore-drill.sh</code> 真实毁库后恢复并校验一致性	已接入 CI
时区正确性	时间戳一律存 UTC，展示按租户时区本地化	跨时区租户各自正确
API 演进	版本前缀 + <code>Deprecation</code> / <code>Sunset</code> 头 + OpenAPI 标注	弃用有宽限期与通告路径

五、大模型的边界：只解释，不决策

这是本系统最需要说清楚的设计。

5.1 边界

所有数值由代码计算，大模型只负责把已算好的结论改写成通顺中文。后端可选 DeepSeek（远程）或 Ollama（本地），未配置时走确定性模板——系统功能完整，只是措辞固定。

5.2 两道代码强制的闸门

闸门	作用	实现
结构校验	返回必须是 JSON 且字段类型正确	<code>_matches_schema</code>
数值一致性	生成文本里每个数字都必须在输入中出现过	<code>_numbers_consistent</code>

任一不过 → 整段作废、退回模板，并记录 `fallback_reason` （`call_failed` / `schema_mismatch` / `number_mismatch`）。

每段输出自带 `llm_used` 与 `model_name`，界面据此区分「算法结论」与「AI 生成解释」。

5.3 实测：幻觉是真实存在的

首次接入 DeepSeek 的真实调用中，模型返回：

「建议物流Agent寻找成本评分**不低于60**的替代运输方案」「接受时效性评分**降低至70**以换取成本评分提升至60」

`60` 和 `70` 输入中根本不存在，系统从未计算过——是模型自行编造的决策阈值。被第二道闸门拦下。

量化（`deepseek-chat`，8 次真实调用）：

提示词状态	被拦截	采用
仅声明「不得新增数字」	7 次 (88%)	1 次 (12%)
补充禁止提出目标值/阈值，并给出定性替代写法	0 次	8 次 (100%)

被编造的清一色是阈值，根因是 `suggested_revision` 这类字段的语义在诱导模型给目标值。**提示词能降低发生率，但不能替代机制**——88% 这个数字本身就是证据。

5.4 严格性取舍

数值校验刻意选严格而非宽松：

- 误判代价：该段文案退回模板，措辞变死板，**数值仍然正确**，`llm_used` 如实标 False
- 漏判代价：AI 解释里的数字与算法结论对不上

两者不对等，因此宁可多退。

六、行业落地实施方案

6.1 分阶段路径

阶段	目标	关键动作	退出标准
一、数据接入	打通主数据	导入物料/供应商/库存快照；ERP 字段映射	初始化待办清零
二、影子运行	不介入决策，只对照	系统出建议，人工照常决策，事后比对	积累 ≥80 条有效历史决策
三、权重校准	用企业自身数据替换专家先验	跑监督式校准，管理员确认	样本外 AUC ≥ 0.55 且通过人工确认
四、辅助决策	进入实际流程	高风险强制人工确认	漂移监控无严重告警

为什么必须有影子期：校准准入门要求 ≥80 条有效样本、正负类各 ≥40。这不是可以跳过的形式——样本不足时系统会明确拒绝校准，维持专家先验。

6.2 部署形态

形态	适用	说明
单机演示	答辩/试点评估	<code>start-demo.ps1</code> 一条命令，SQLite，前端由 FastAPI 托管
单实例生产	中小企业	PostgreSQL + Docker，Alembic 迁移
多实例	高可用	当前不支持，见 6.3

6.3 已知限制（落地前必须知情）

诚实列出，不藏：

1. **模型注册表仍是本地文件**（`src/model_registry.py`）：JSONL 全量读写、无上限。并发丢写已修复——读-改-写整体加文件锁、落盘走临时文件 + `os.replace()` 原子替换（`src/file_store.py`），回归见 `tests/test_file_store_concurrency.py`。

修复前实测：24 线程并发 `register()` 只活下来 1 条，另有 2 行 torn write 被 `load_all()` 当坏行静默跳过，全程零异常。此前文档给的缓解措施“别用 `--workers >1`”是**无效的**：这两个端点是同步 `def`，FastAPI 把它们放进 `anyio` 线程池执行，单个 worker 内部就并发。

仍然成立的限制：容器重启丢数据，不进数据库也就不进备份；文件锁不跨主机，多机共享存储部署仍需迁移到数据库。

2. **tenants 无时区列**：「今日出入库」这类指标直接取 UTC，会让 Asia/Shanghai 的企业每天 08:00 才翻页，早班数据算进前一天。跨时区租户上线前需补。
3. **除 PostgreSQL 专项用例外，其余测试跑在 SQLite 上**：SQLite 不校验外键、约束语义与 PG 不同。生产用 PG 目前是声明，不是被持续全量验证的事实。
4. **算法可信度仍有开放问题**：权重校准闭环能否真正提升成功率（F1）、embedding 检索是否优于 TF-IDF（F3），均**未取得可证伪的实验结论**，对外不作承诺。

七、可复现性

本项目对外的核心承诺是**任何数字都能现场跑出来**。

```
# 完整演示（依赖→迁移→播种→构建→启动，单进程单端口）
.\start-demo.ps1

# 后端全量测试
cd ChainGuard; pytest -q

# 端到端验收门禁（跳过必须显式申报，不允许静默跳过）
cd chainguard-web; npm run test:e2e:gate

# 本文第三节的性能数字
python -c "from src.benchmark import run_baseline, run_ablation; print(run_baseline()); print(run_ablation())"
```

验收门禁按套件读 Playwright 的 JSON reporter 判定，**跳过数必须与** `run-acceptance-gate.mjs` **中申报的** `expectedSkips` **一致**，否则判失败——避免「用例没跑」和「用例通过」显示成同一个绿灯。